Natural Disaster Survival — App Store readiness

Branch `claude/nat-disaster-app-store-launch-cwrjpa` · 21 August 2026

The game now has its own door, its own build, and a test that can tell whether either one is broken. Three things it did not have: a match that survives ninety seconds, a first screen that does not wait on five other games, and two clients that agree on what happened.

The numbers

	BEFORE	AFTER
Script files loaded	553	**83 on the web page · 1 in the app**
JavaScript shipped	25.0 MB	**4.25 MB on the web page · 1.69 MB minified in the app**
World built at launch	the city	**the island**
Network needed to play	Google Fonts + a CDN decoder	**none at all**
Match survives	~90 seconds, then a crash	**no known limit**
Two clients agree for	0 ticks	**5,400 ticks (90 s, 32 bots)**
Time to a playable match	~6 s	**2.1 s**
Total app payload	—	**19.5 MB (17.7 of it audio)**

`Timings measured headless on SwiftShader, which is not a phone. Compare runs; do not quote one.`

1 · The crash

> LIVE ON MAIN BEFORE THIS BRANCH
>
> **Every survival match died after about ninety seconds.** `city/marine_predation.js` and `city/wildlife_orca.js` both wrap `CBZ.sharkBrain`, and both re-armed from a per-frame pass asking “am I the outermost link?”. Each frame, each file saw the other's wrapper on top, concluded it had been displaced, and wrapped again — two new closures per frame, for ever. A few thousand links deep the next call died with `RangeError: Maximum call stack size exceeded` inside the wildlife tick.
>
> The question a link should ask is “am I in the chain at all?”. Each wrapper now carries its owner and its fall-through and walks the chain looking for itself. Ratcheted at three links, measured after 16,000 simulated ticks.

It was found by the test built for this wave, not by reading the code — which is the case for most of what follows.

2 · Loading

`index.html` is six games: the prison, the city, the campaign, the casino, the aircraft, the elections. Natural Disaster Survival is one of them, and on a phone it paid for all six before it could draw a frame.

- **`disaster.html`** — the same page minus the scripts this game never runs, opening straight onto the island. Generated from `index.html` rather than hand-written, so the HUD markup, the CSS and the boot order keep one source of truth.
- **The list is measured, not guessed.** `tools/disaster-minimize.mjs` drops groups of files and asks the game whether it still boots, still runs all eleven disasters, and still has every named system on `CBZ`. A file leaves the build only when the game passes without it.
- **The app gets one file.** The kept scripts are concatenated in page order and minified: 1.69 MB, one fetch, one compile, instead of 556 of each. Each block keeps the failure isolation a script tag has, and `document.currentScript` is stood in for per file so path-resolving code behaves exactly as it does unbundled.
- **The island reports its own progress.** Six `CBZ.bootStep` checkpoints, so the loading meter — which already draws on a worker thread precisely so it can count through a frozen main thread — has something to count.

> WHAT THE SEARCH TAUGHT US
>
> Left alone, the file search took the build to **51 files** — and that was a confession, not a triumph. A test can only defend what it looks at, and this one dropped the collision system (the census named a symbol another file publishes), the mode's own HUD (it named a DOM element that is in the HTML either way), the loading meter, the static-batching pass, all nine facade grammars and the capability bus. Nothing *failed*. The phone would just have melted, undressed, with no progress bar and nobody able to vault.
>
> So the census rows now name what each *file* publishes, the oracle holds W for a second and checks the player actually walked, and 33 files are **pinned** — declared rather than proved, because a headless run has no frame budget, no thumbs and no eyes.

3 · Offline

An App Store app may not fetch what it needs from a third party at run time, and has to work in aeroplane mode the first time it is opened. Two things did:

- `city/playercars.js` pulled the **Draco decoder from unpkg.com** when the first Ferrari loaded. Vendored to `src/vendor/draco/`.
- `index.html` loaded the **UI typeface from Google Fonts** as a render-blocking stylesheet. The 29 KB latin subset now ships in the repo (OFL, recorded in `assets/LICENSES.md`).

The build copies only what the code actually asks for, and the whole tree is verified by booting it and running the full disaster roster against it.

4 · The app is interrupted; a browser tab is not

iOS backgrounds the app for a phone call and drops its WebGL context under memory pressure. three.js rebuilds its own GPU resources, but nothing told the *game*: a hundred survivors, eleven disasters and a drowning player kept running behind a black screen, and the player came back to a death they never saw. `systems/glcontext.js` pauses the match on context loss and on losing the foreground, resumes the audio graph on the way back in, and asks for the context back if it has not returned in four seconds.

5 · Ready for multiplayer — the part that is not a transport

Multiplayer is a determinism problem before it is a networking problem. Two machines have to produce the same island, the same arc, the same wave and the same hundred bodies from one seed, or every design above — lockstep, rollback, server-authoritative — is built on sand. `tools/determinism-check.mjs` boots the game twice in two browsers and compares every body every sixty ticks. It started at **zero** ticks of agreement.

WHAT WAS WRONG	WHY IT COULD NEVER HAVE WORKED
The hazards rolled `Math.random`	Where the lightning lands, which way the tsunami comes in, which buildings the quake takes — all of it different on every client.
Bot think cadence came off the camera	A bot's *decisions* depended on where the local player was looking; two cameras put the same bot on opposite sides of an LOD boundary.
The think schedule counted from page load	Which bots thought on tick 1 depended on how long the title screen had been up.
The clock moved by the wall	Every cooldown advanced by however long that machine's last frame took.

All four are fixed, plus a fixed 60 Hz step (survival only, one-line revert) and `net/survnet.js`: stable ids, the whole match in one 2.5 KB ArrayBuffer and back, and one fingerprint two clients can compare. No socket, no lobby, no prediction — those depend on a transport decision nobody has made yet. **Result: 5,400 ticks — ninety seconds, 32 bots, several disasters — identical to the millimetre across two browsers.** That is on `index.html`; the sliced page still diverges inside the first second, which is the one thing this wave leaves open and the tool now names precisely.

6 · What is on the Mac side

Nothing in this branch is Swift. The iOS side is Capacitor — a config file, generated art, a privacy manifest, plist keys and a runbook. The Xcode project is generated on your machine and is regenerable at any time.

- `npm run build:ios` — writes `dist-ios/www`
- `npm run check:ios` — boots that bundle, runs all eleven disasters
- `npx cap add ios` — generates the Xcode project
- `node tools/setup-ios.mjs` — plist keys, privacy manifest, icon, splash
- `npx cap sync ios` — after every rebuild
- `ios/GO-IOS.md` — the full runbook

The runbook also carries the answers App Review will want in advance: why this is not a repackaged website (guideline 4.2), the age rating this game honestly needs, the privacy label, and a device checklist for the things a headless test cannot see.

7 · What is not done

- **Nothing has run on a device.** Every number here is headless Chromium on software rendering. The first real measurement is a cold launch on your phone.
- **Multiplayer is not built** — deliberately. What exists is the ground it needs.
- **The payload can go further.** The search is bounded by what the test can prove; a stricter or longer search finds more.
- **Determinism is proved on the full page, not on the sliced one.** `index.html` runs 5,400 identical ticks; `disaster.html` — the page the app ships — still diverges by millimetres within the first second, so something in the files the slice drops is holding the full page steady. That is the next thread to pull, and `tools/determinism-check.mjs --every 1` names the actor and the axis.

- **Ninety seconds is not a whole match**, and neither run had a second human moving in it.

Every claim in this document is reproducible from the repository:
`node tools/disaster-check.mjs` · `node tools/determinism-check.mjs` · `node tools/disaster-minimize.mjs --verify` · `npm run check:ios`